



ĐỒ ÁN CHUYÊN NGÀNH
C04337 | HK252

Ứng dụng DevOps và container hóa trên Kubernetes cho hệ thống Moodle LMS tích hợp AI

GVHD: TS. Nguyễn Quang Hùng

SVTH1: Trần Phước Nhật - 2212412

SVTH2: Nguyễn Đăng Cường - 2210432

Ho Chi Minh, 07/06/2026

Table of CONTENTS

- 1 Tổng quan đề tài & xác định vấn đề
- 2 Cơ sở lý thuyết & lựa chọn công nghệ
- 3 Xây dựng & thiết kế hệ thống
- 4 Kết quả & hướng phát triển
- 5 Demo



Tổng quan đề tài & xác định vấn đề

Giới thiệu tổng quan

- Bối cảnh: Xu hướng chuyển đổi số trong giáo dục
- Nhu cầu: Hệ thống LMS linh hoạt, có khả năng mở rộng
- Giải pháp: Moodle trên Kubernetes với AI tích hợp
- Công nghệ: Cloud-native, containerization, automation

Welcome to the Moodle community

The place to get support, ask and answer questions and contribute to the open source learning platform, Moodle LMS.

Get involved →



54,140,810+
Courses



151,956+
Sites



236+
Countries

Mục tiêu

1. Triển khai Moodle LMS trên Kubernetes

- Khả năng chịu tải tốt
- Khả năng mở rộng tự động (auto-scaling)
- High availability và fault tolerance

2. Xây dựng CI/CD pipeline hoàn chỉnh

- Tự động hóa build, test, deploy

3. Tích hợp AI subsystem

- Local AI (Ollama) hoặc Cloud AI (OpenAI, Gemini, ...)
để hỗ trợ học tập

Phạm vi

Multi-cloud (DigitalOcean, AWS, Azure)

Moodle LMS



Hệ thống quản lý học tập mã nguồn mở

Ưu điểm:

- Linh hoạt, dễ tùy chỉnh
- Cộng đồng phát triển lớn
- Hỗ trợ đa dạng plugins và themes

Phiên bản sử dụng: Moodle 5.x

Stack: PHP 8.2-FPM, PostgreSQL 15, Redis

2

Cơ sở lý thuyết & lựa chọn công nghệ

Nền tảng điều phối container

Bảng so sánh nền tảng điều phối

Tiêu chí	Dịch vụ của Cloud (VD: ECS, ACA, CloudRun)	Kubernetes
Tính độc lập	Phụ thuộc Cloud. Tốn kém nếu đổi nền tảng.	Độc lập. Dễ dàng đổi sang Cloud khác hoặc On-Premis Server.
Nhân sự vận hành	Tối thiểu Cloud Provider lo phần lớn quản lý.	Cao Buộc phải có kỹ sư giỏi để vận hành, bảo trì.
Khả năng tùy biến	Giới hạn tùy vào Cloud Provider.	Không giới hạn. Có thể tích hợp thoải mái.
Tốc độ triển khai	Rất nhanh. Phù hợp ra thị trường ngay lập tức.	Chậm ở giai đoạn đầu Mất thời gian thiết kế và xây dựng kiến trúc.

Infrastructure as Code (IaC)

Quản lý infrastructure bằng code

Terraform: Công cụ IaC đa nền tảng

Lợi ích:

- Version control cho infrastructure
- Reproducibility: Tái tạo môi trường dễ dàng
- Automation: Giảm thiểu lỗi thủ công



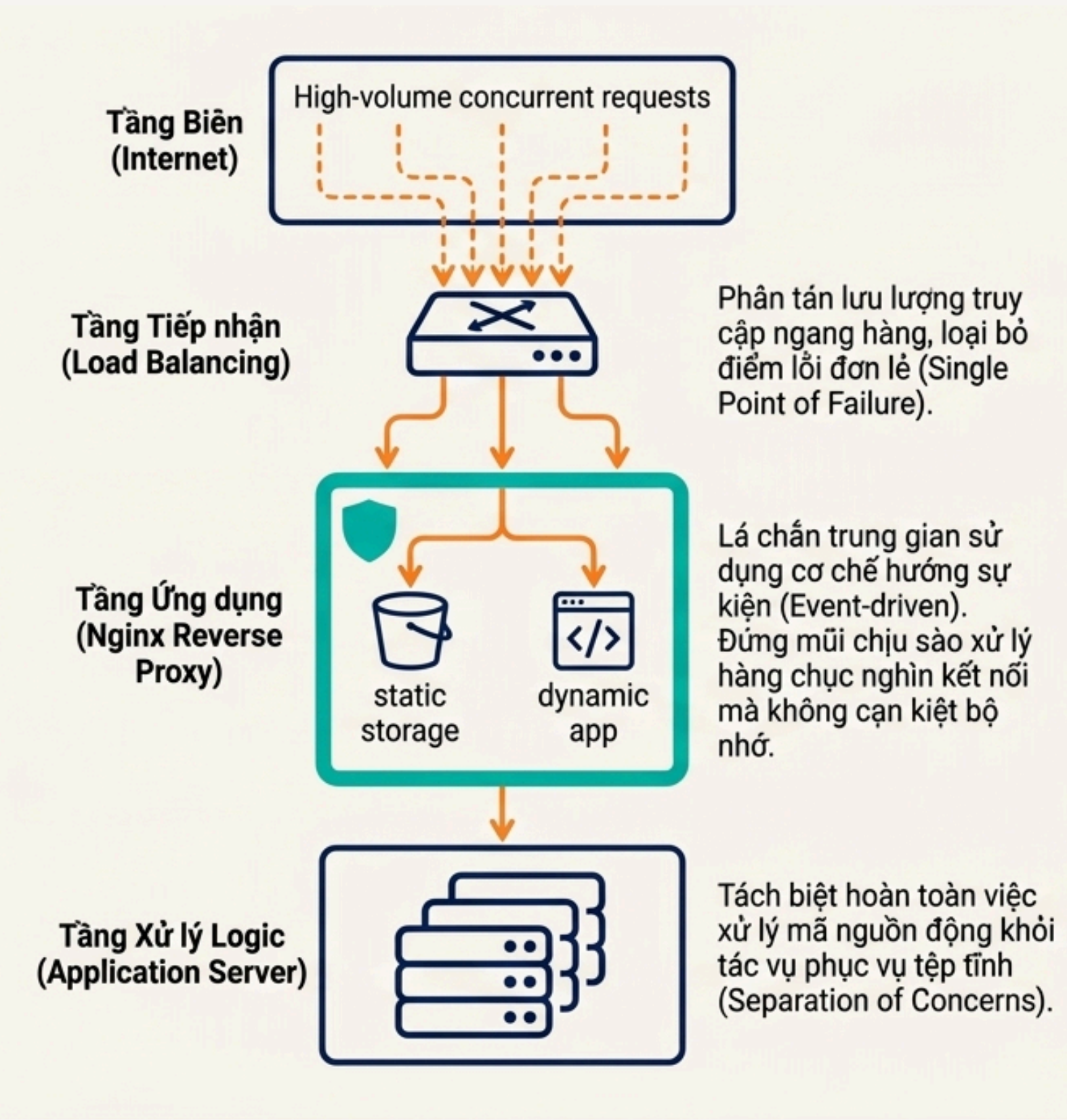
Phân tích chi phí hạ tầng

Cấu hình chung

- Autoscale 3-6 Node
(Mỗi node 4VCPU, 8GB RAM)
- 2 Database thiết kế theo
Primary + Standby.
(Mỗi DB 2VCPU, 4GB RAM, max
97 connection pool)

Tiêu chí	DigitalOcean (DO)	AWS	Azure
K8s Basic	0\$/month	73\$/month	0\$/month
Node (Scale 3->6)	48\$/month (144\$-288\$)/3-6 month	~124\$/month (372\$-744\$)/3-6 month	~124\$/month (370\$-740\$)/3-6 month
Database	\$122/month	~\$150/month	~\$140/month
Load balancer	\$15.00/month	~\$30.00/month	~\$20.00/month
Tổng	~281 - 425\$	~625 - 997\$	~530 - 900\$

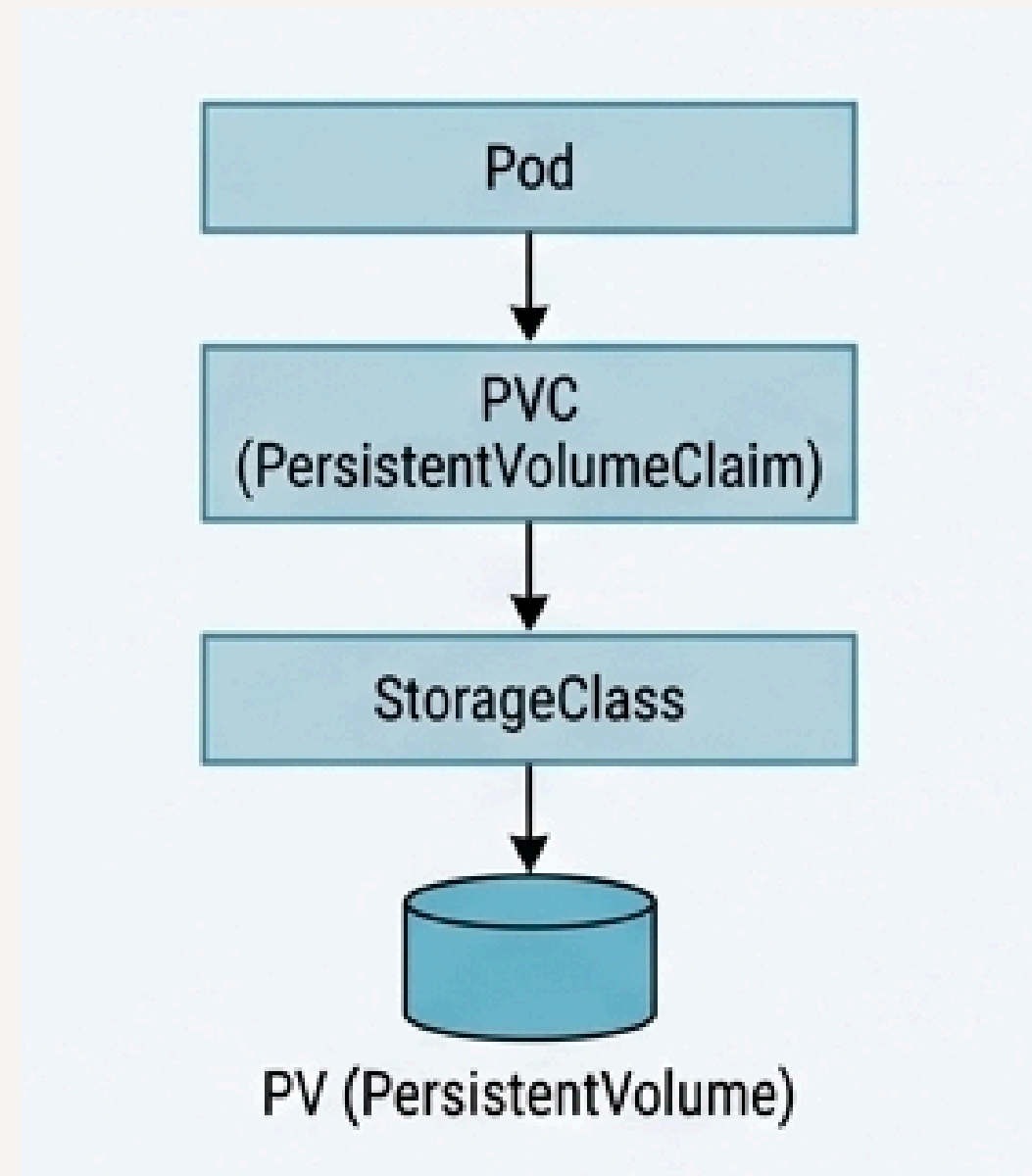
Kiến trúc phân phối tải & Nginx reverse proxy



Bảng đánh giá kiến trúc phân tán và nguyên khối

Tiêu chí	Nginx Reverse Proxy	Web server nguyên khối (Apache mod_php)
Cơ chế I/O	Hướng sự kiện, Non-blocking	Dựa trên tiến trình, Blocking
Quản lí kết nối	1 worker xử lí nhiều kết nối đồng thời	Tạo 1 thread/process mới cho mỗi kết nối
Xử lí nội dung	Tách biệt: Nginx trả file tĩnh, App server xử lí logic động	Xử lí hỗn hợp: Một tiến trình lớn gánh mọi request
Tiêu thụ tài nguyên	Tối ưu: RAM duy trì ổn định	Tiêu tốn RAM tuyến tính
Tính mở rộng	Co giãn Độc lập từng thành phần	Khó mở rộng Tỉ lệ bottleneck cao

Lưu trữ dữ liệu phân tán



- Tách biệt vòng đời
- Truy cập đồng thời (RWX)

Bảng so sánh lưu trữ phân tán Longhorn vs. NFS

Tiêu chí	NFS truyền thống	Longhorn phân tán
Kiến trúc	Máy chủ bên ngoài	Phân tán trên các node
Điểm lỗi duy nhất (SPOF)	SPOF cao	Không có SPOF
Độ trễ và hiệu năng	Độ trễ cao	Độ trễ thấp
Nhân bản dữ liệu	Không sao chép sẵn	Có bản sao
Mở rộng & tận dụng tài nguyên	Hạn chế bởi 1 máy chủ	Kết hợp lưu trữ từ tất cả node
Phù hợp với cụm nhiều node	Điểm nghẽn	Lý tưởng/Sẵn sàng cao

Caching

Bảng phân tích đánh giá

Tiêu chí	OPcache	APCu	Redis
Bản chất	Trình lưu trữ mã trung gian	Bộ nhớ đệm đối tượng cục bộ	Lưu trữ Key - Value phân tán
Phạm vi	Cục bộ (Pod)	Cục bộ (Pod)	Dùng chung (Toàn hệ thống)
Độ trễ	Tức thời	Cực thấp ~ 0.001ms	Thấp ~ 0.5-1ms
Cơ chế loại bỏ	Reset hoặc Overwrite khi đầy	Dựa trên time sống (TTL)	LFU cho Cache Noevition cho Session
Dữ liệu lưu trữ	Code PHP đã biên dịch sẵn	Dữ liệu cấu hình thường xuyên truy cập	Dữ liệu trong DB, Phiên đăng nhập
Quyết định sử dụng	Bắt buộc kích hoạt để tối ưu CPU	Cache L1 giảm overhead mạng	Cache L2 đồng bộ dữ liệu khi nhiều pod chạy song song

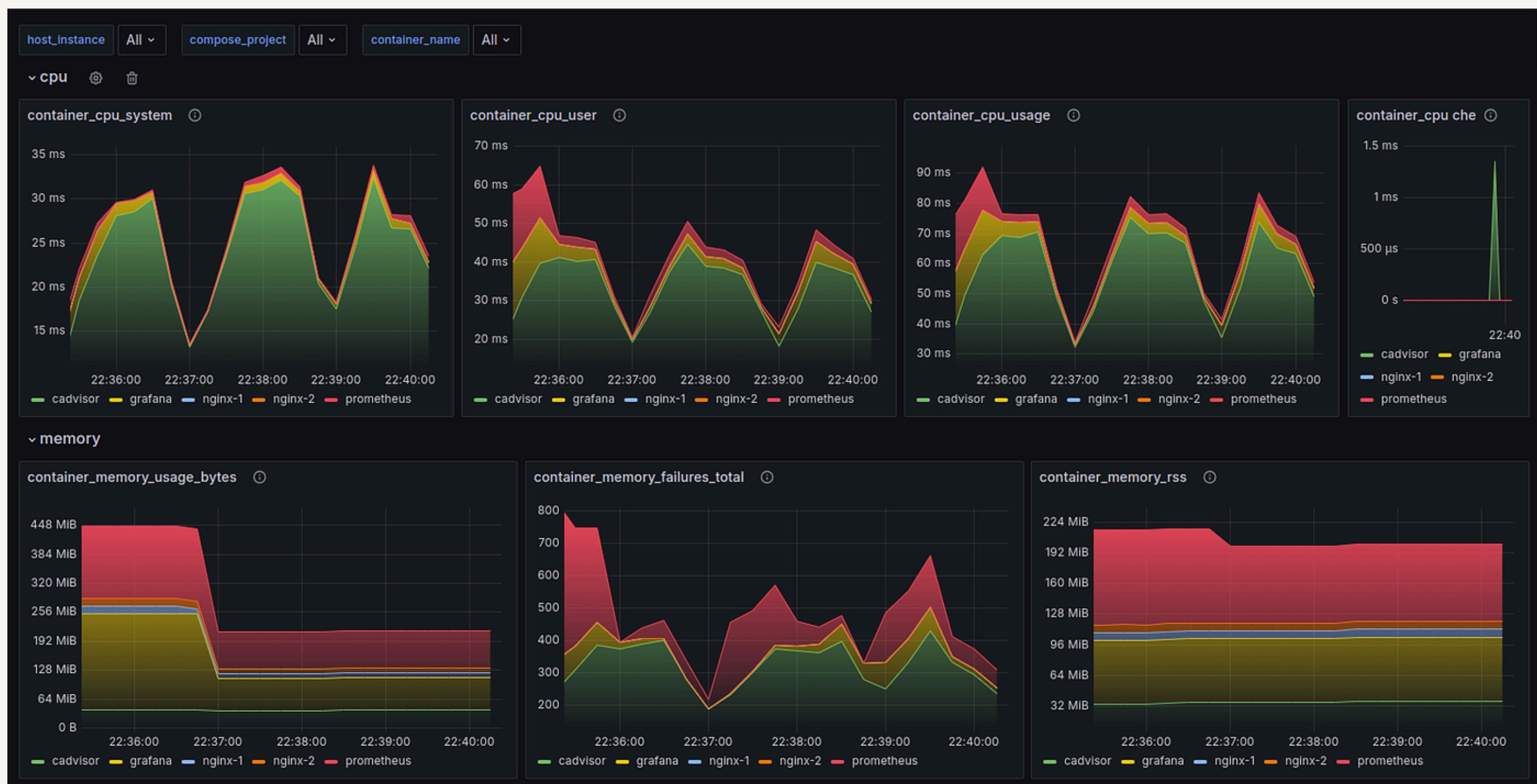
Package Manager

Helm là package manager tiêu chuẩn cho k8s, giúp đóng gói toàn bộ tài nguyên phức tạp của 1 ứng dụng thành 1 đơn vị triển khai duy nhất gọi là Chart

Tiêu chí	Manual YAML	Helm
Quản lý tài nguyên	Phân tán	Tập trung
Khả năng tái sử dụng	Thấp Phải tạo mới nếu thay đổi hạ tầng	Rất cao Dùng chung template
Thao tác triển khai	Chạy lệnh cho từng file hoặc thư mục	Chỉ 1 lệnh duy nhất
Quản lý phiên bản	Khó khăn Tự quản lý thay đổi	Tích hợp sẵn Quản lý revision hỗ trợ Rollback tự động
Mức độ phù hợp	Dự án nhỏ, kiến trúc đơn giản	Hệ thống phức tạp

Hệ thống giám sát

Dùng Prometheus và Grafana để giám sát hệ thống trực quan



3

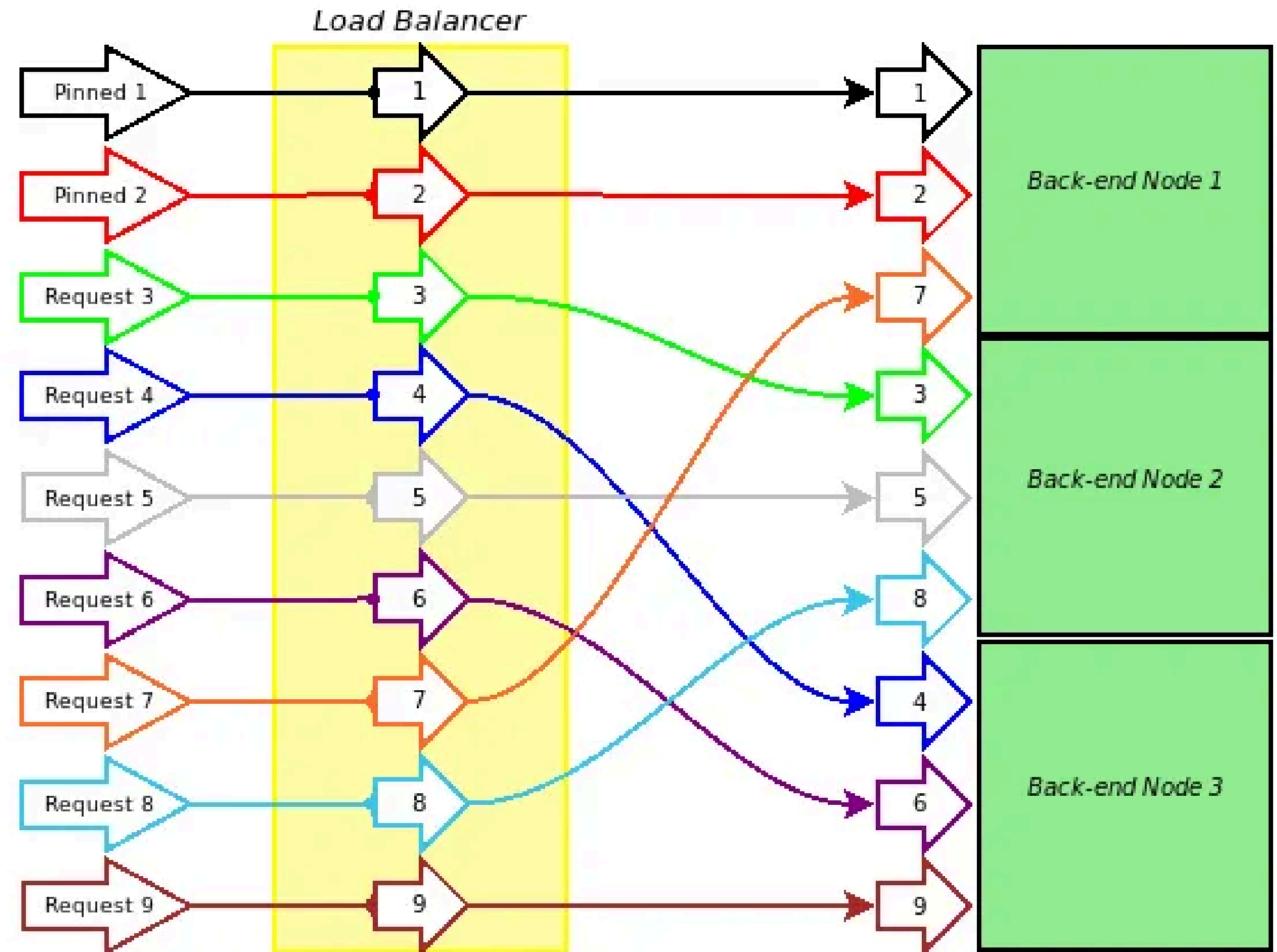
Xây dựng & thiết kế hệ thống

Ingress Controller

Nginx

- Thuật toán Least Connections thay vì mặc định Round Robin.
- Vì trong Moodle các request có thời gian lệch khá lớn, nên cần cân bằng giữa các pod.
- Nhằm tránh tình trạng pod rảnh rồi, pod hoạt động quá mức

Least Connections Load Balancing with x2 pinned connections



Database

PostgreSQL

- Master - Replica (Write - Read)
- Sử dụng session mode thay vì transaction
- Thiết lập Pgouncer để giữ kết nối ổn định

 **moodle-db-postgres-production-87644**
in [first-project](#) / 4 GB RAM / 2vCPU / 60 GiB Disk / Primary + Standby / SGP1 - PostgreSQL 16

[Upsize database cluster](#) [Actions](#)

[Overview](#) [Insights](#) [Logs & Queries](#) [Users & Databases](#) **[Connection Pools](#)** [Network Access](#) [Settings](#)

Connection Pools

[Learn](#) [Create a Connection Pool](#)

Available backend server connections: 12 / 97 [?](#)

Pool Name	Database	Username	Pool Mode	Size	Connection details
moodle-pool-production	moodle	moodleuser	session	85	Connection details ...

Settings

[Learn](#)

Project

 first-project [Edit](#)

Configuration

 **Compute** - Basic - Shared CPU / 2 vCPU / 4 GB RAM / Connection limit: 97 [Edit](#)

 **Storage** - 60 GiB SSD

 **Standby** - One additional node

 **Total Cost** - \$121.80/month

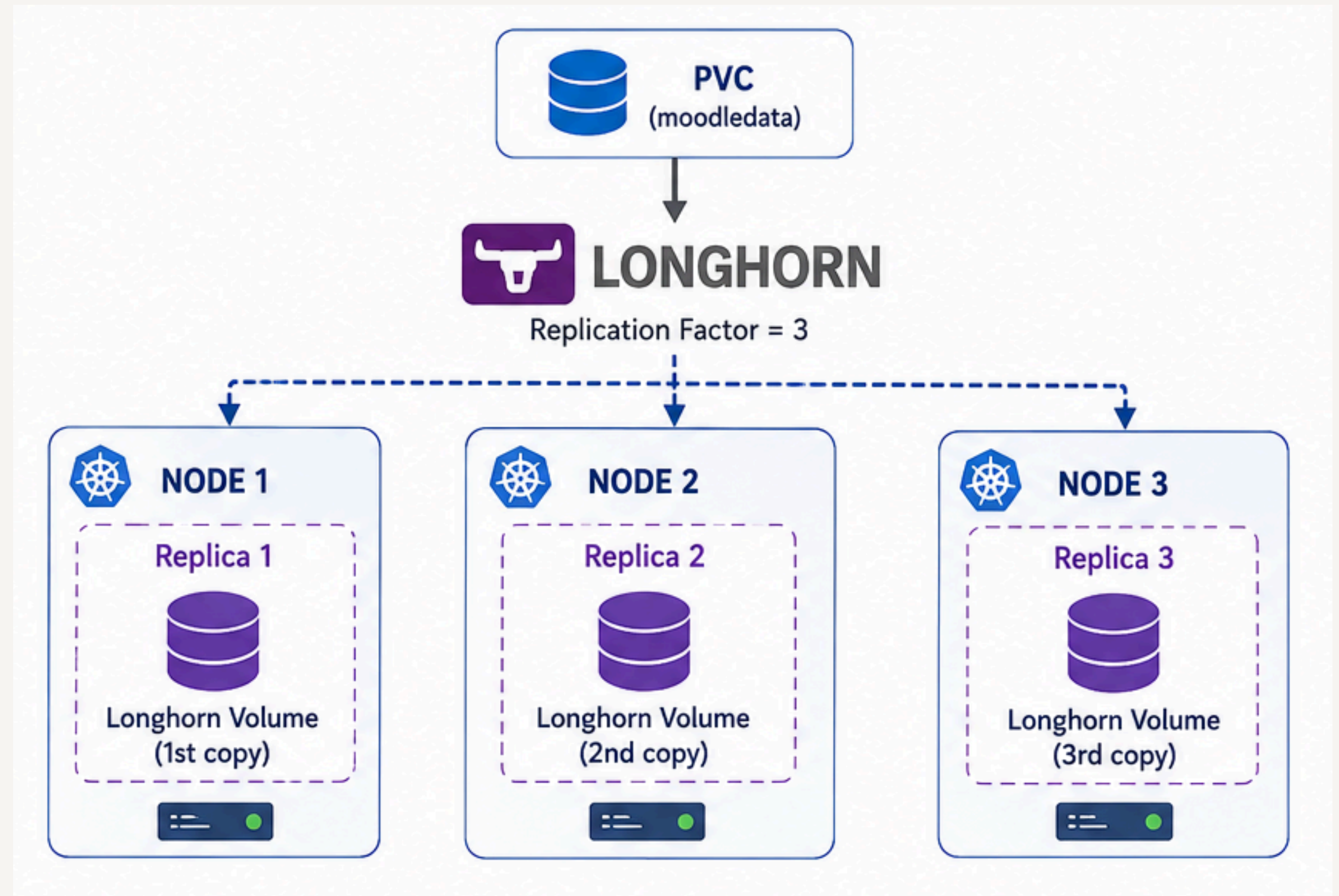
Cluster datacenter

 **Singapore** • Datacenter 1 • SGP1 [Edit](#)

VPC network - [default-sgp1](#)

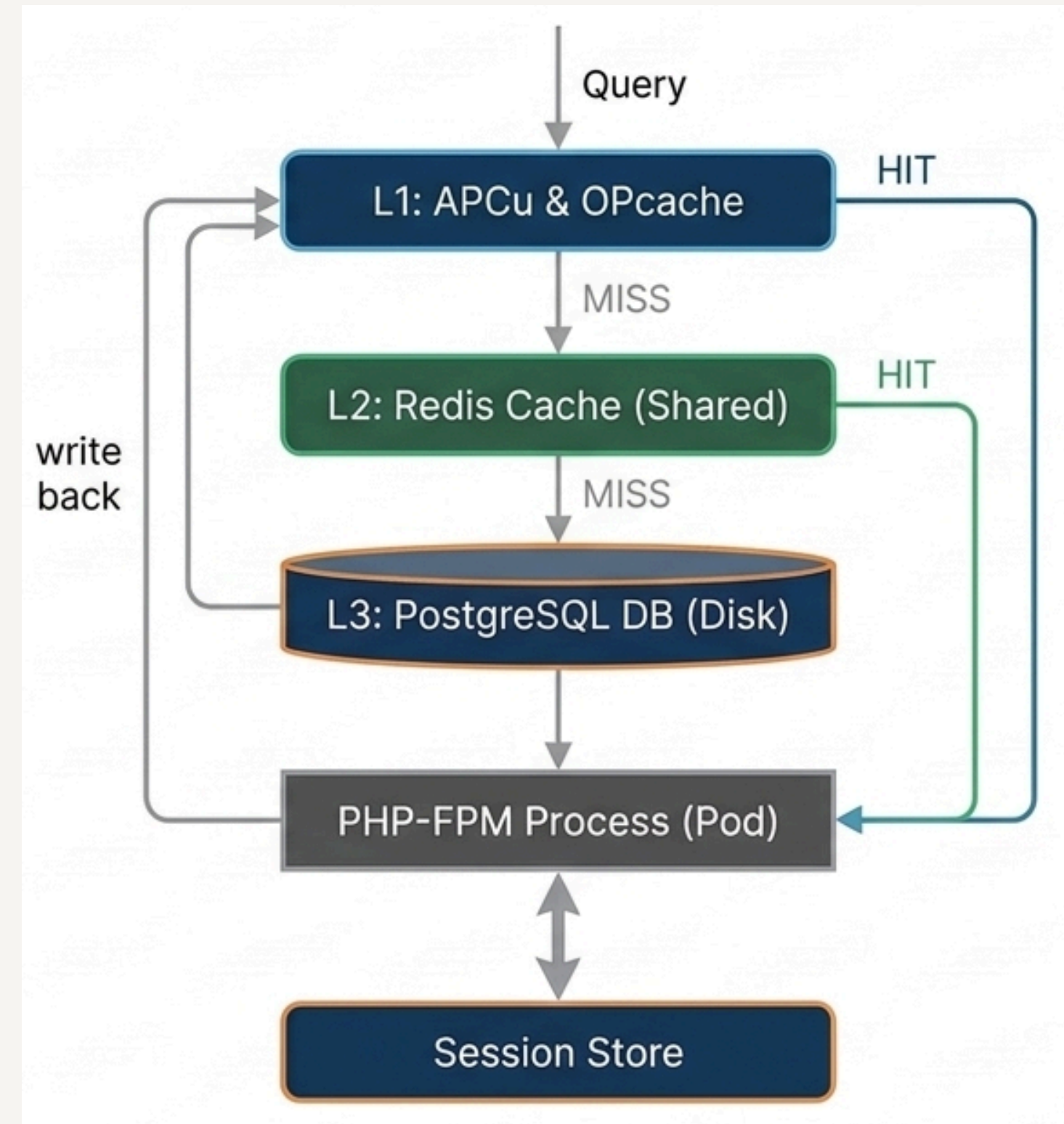
Config & Storage Resources

- Longhorn:
 - *Dữ liệu nặng, file lớn.*
- Local pod:
 - *Dữ liệu tĩnh (html, css,...)*
 - *PHP OPcache*
 - *APCu*
 - *NGINX config*
 -
- ConfigMap/ Secret:
 - env config, sensitive data



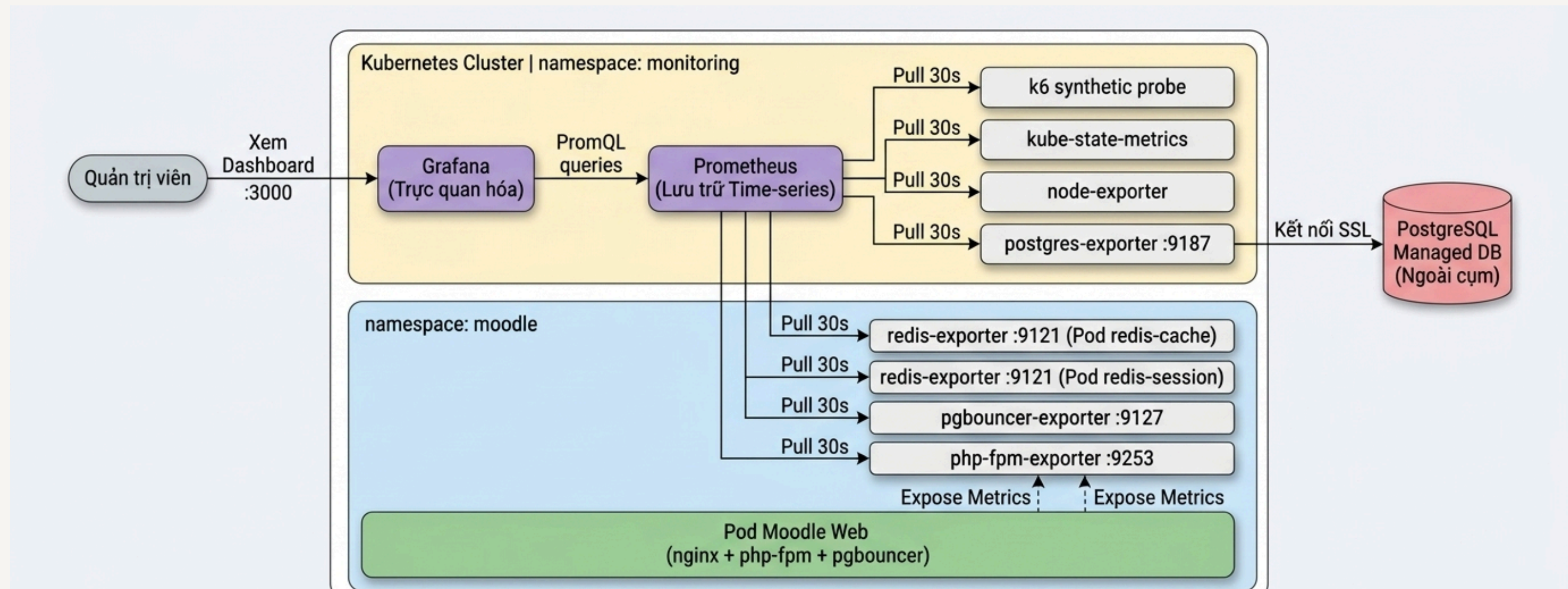
Caching

- **L1 (mỗi pod 1 cái, RAM):**
 - APCu cache: dữ liệu tính toán như string, biến,....
 - Opcode cache: mã nguồn tải trang (byte code)
- **L2 (chung cho các node, RAM):**
 - Redis cache: dữ liệu mà APCu miss.
- **L3 (disk):**
 - DB: Dữ liệu toàn bộ hệ thống
- **Session Store: Phiên đăng nhập người dùng**



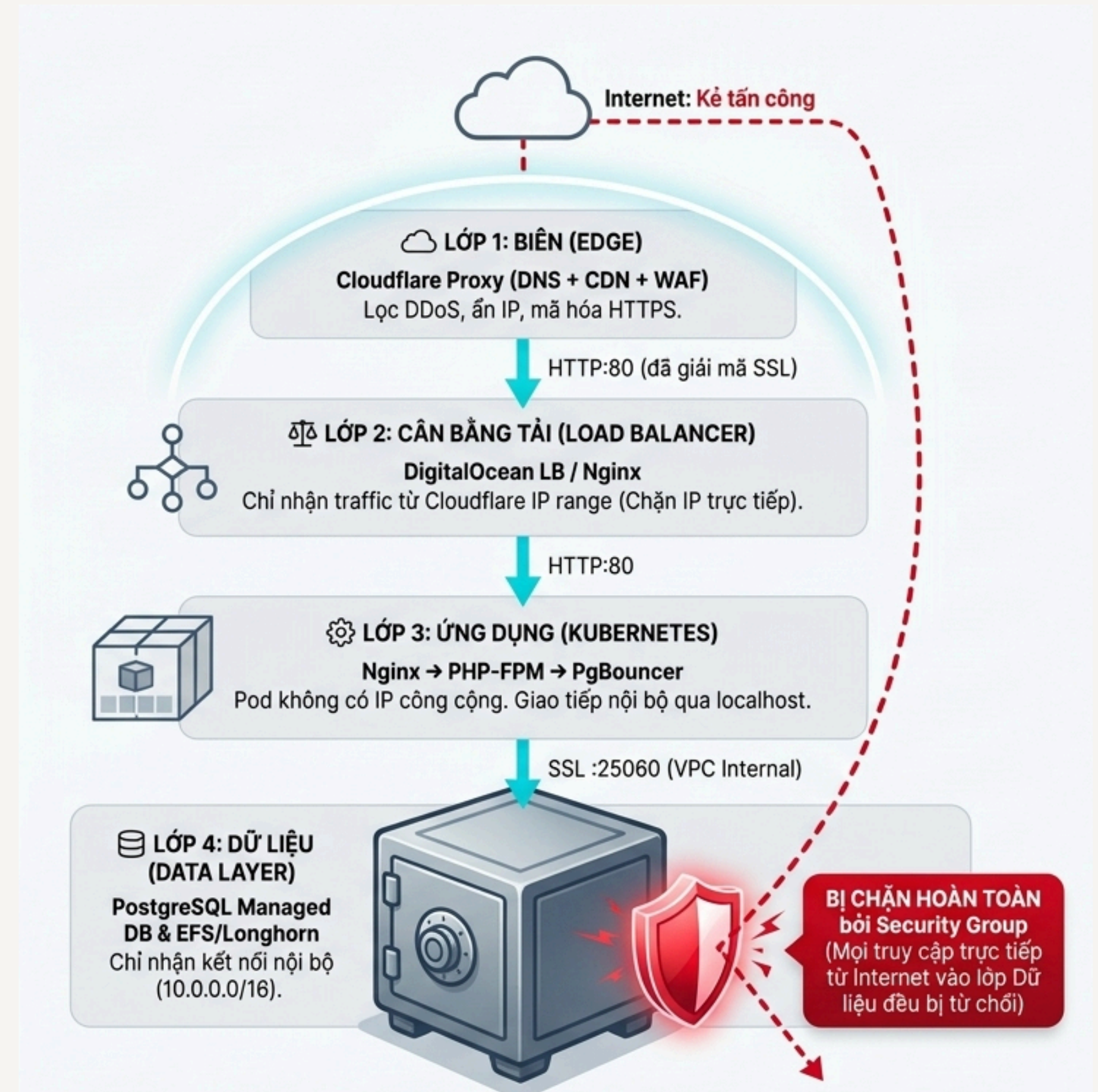
Metric & Giám sát

- Chia grafana thành các dashboard theo mục đích sử dụng.
- Mỗi pod service đều sử dụng exporter để trả về dữ liệu metric

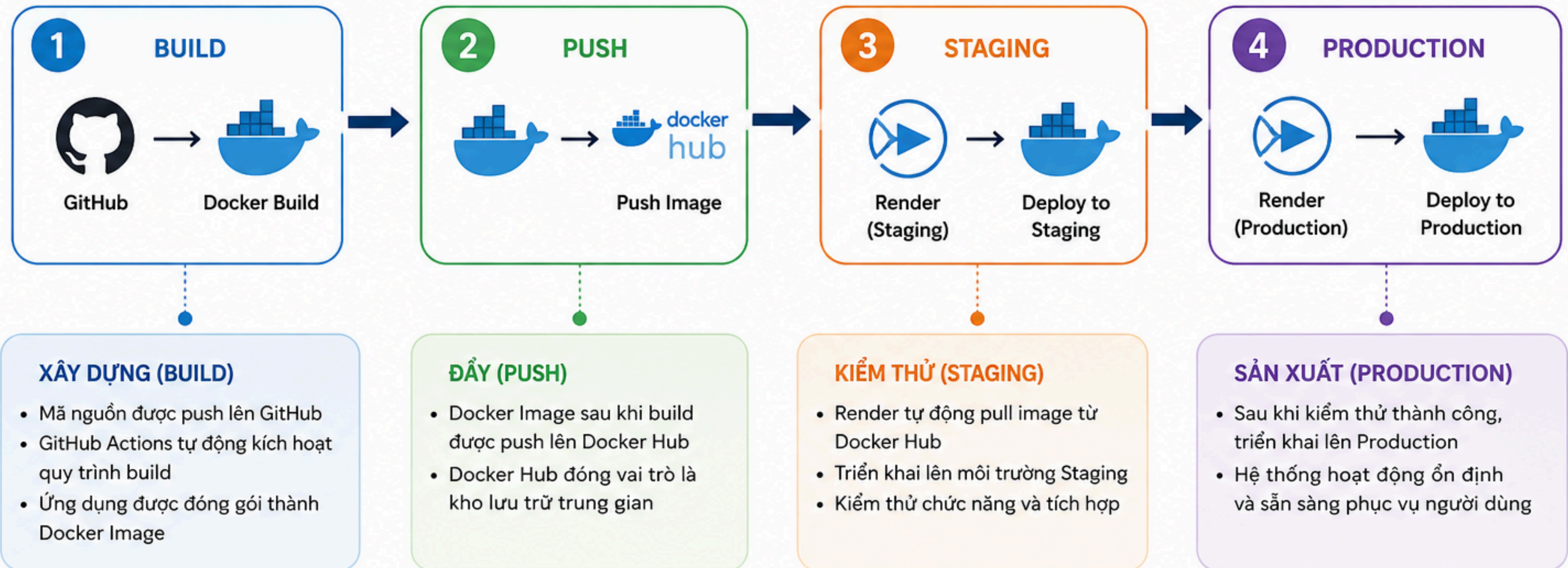


Security

- Phần lớn các vấn đề bảo mật được Cloudfare xử lý (các cuộc tấn công vào hạ tầng như DDoS)
- Moodle có sẵn khả năng bảo mật, không cho phép truy cập qua ip server mà chỉ qua domain.
- Các nơi quan trọng như database chỉ cấu hình cho phép truy cập nội bộ thông qua K8s



Hiện thực CI/CD

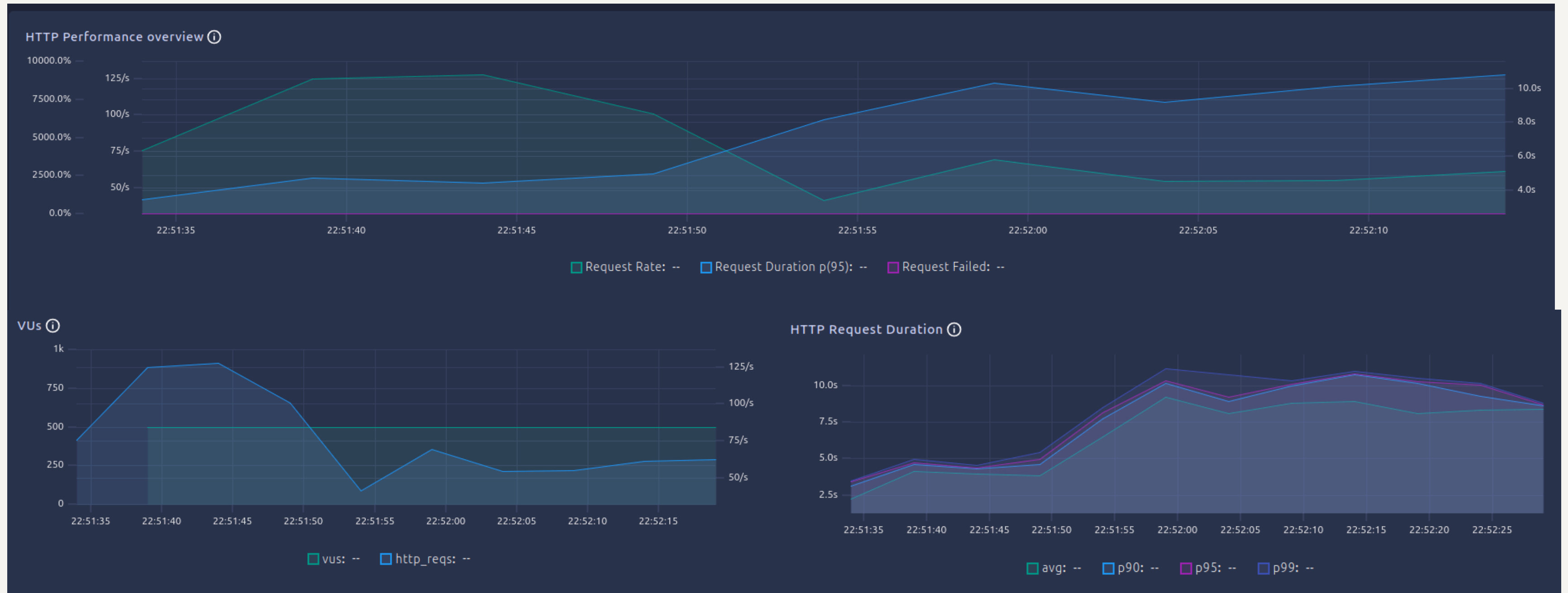


4

Kết quả & hướng phát triển

Khả năng chịu tải

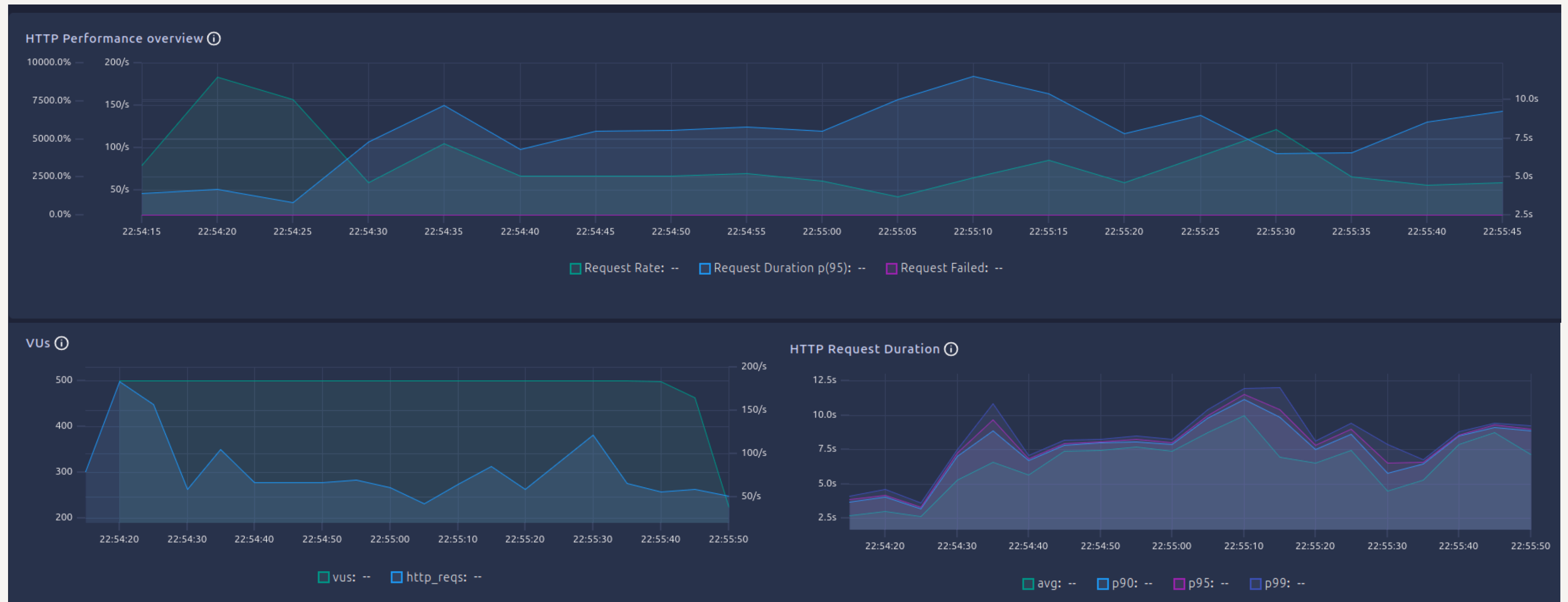
Kịch bản: 500 User truy cập vào bài thi, sau 30s cùng lúc nộp bài thi



30s đầu tiên: user truy cập vào bài thi và điền sẵn đáp án

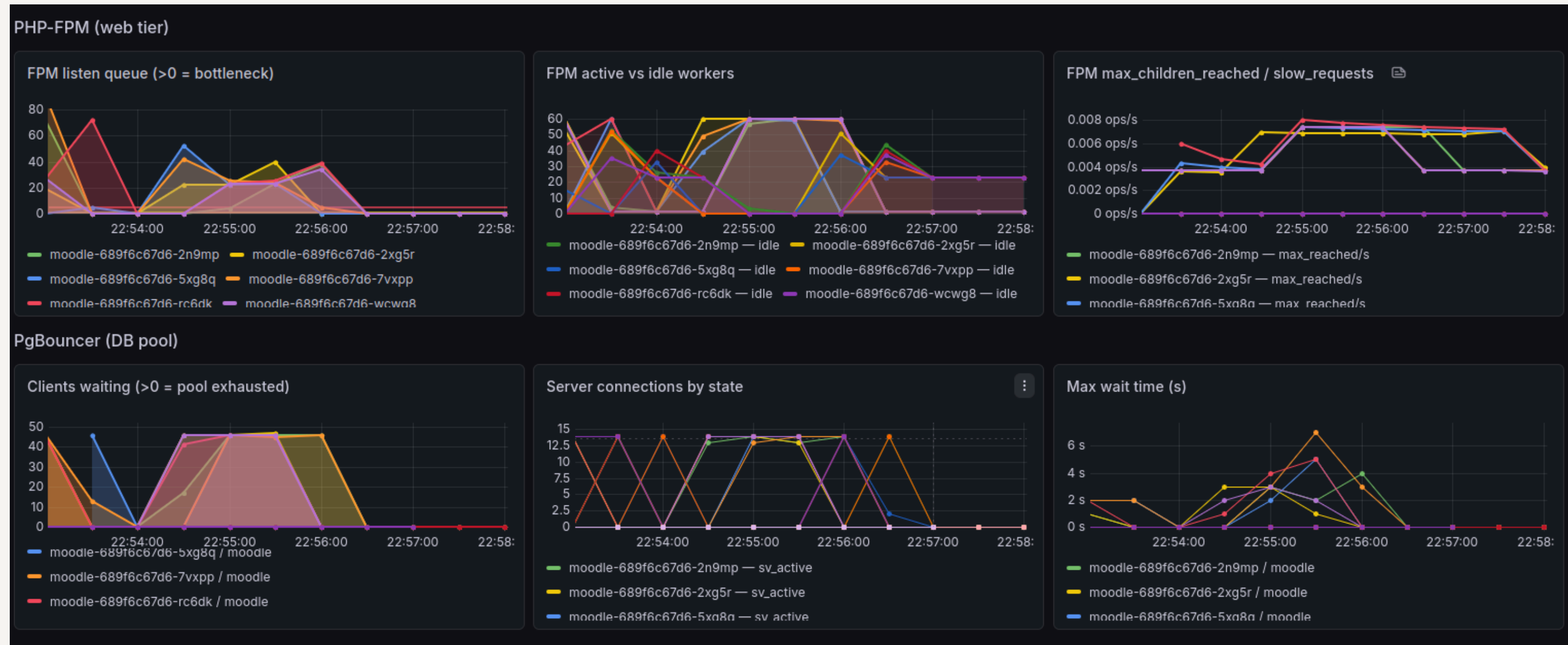
Khả năng chịu tải

Kịch bản: 500 User truy cập vào bài thi, sau 30s cùng lúc nộp bài thi



Sau 30s: user đồng loạt nộp bài thi cùng lúc

Dashboard Grafana



Kết luận:

- Bottleneck nặng nhất là Database. (PgBouncer Clients waiting ~50)
- Các query xếp hàng đợi được DB xử lý làm nghẽn tới tầng App (FPM listen queue ~40)

Dashboard Grafana

Redis (session & cache)

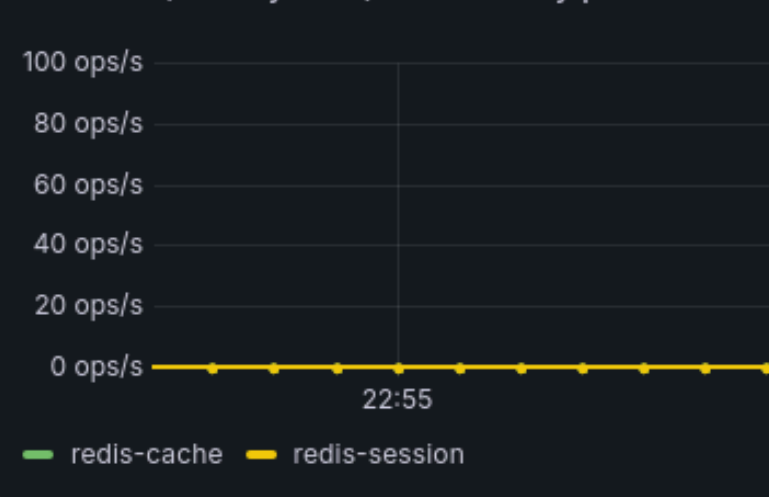
Redis PHP ses...



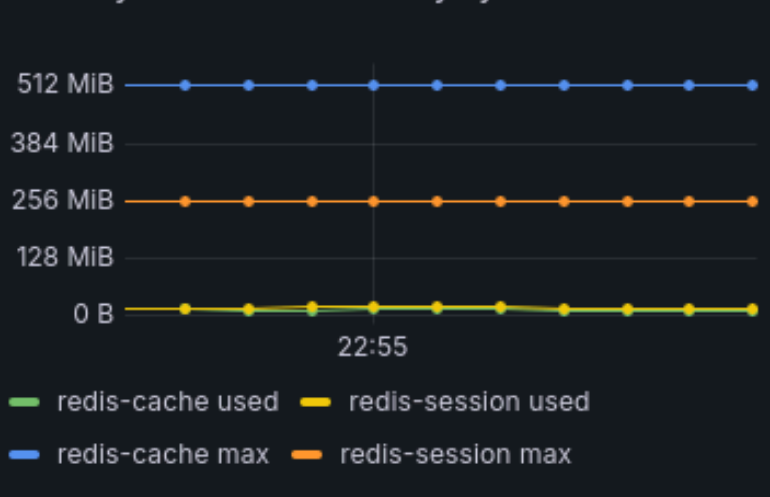
Redis L2 app c...



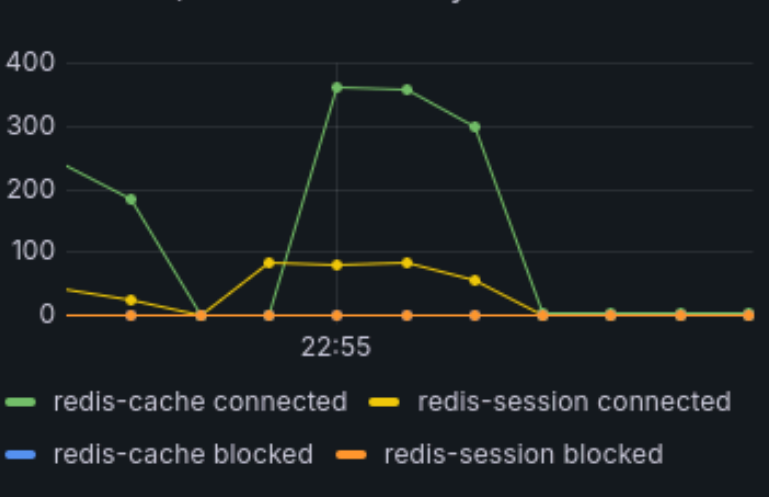
Evictions / sec by role (>0 = memory pres...



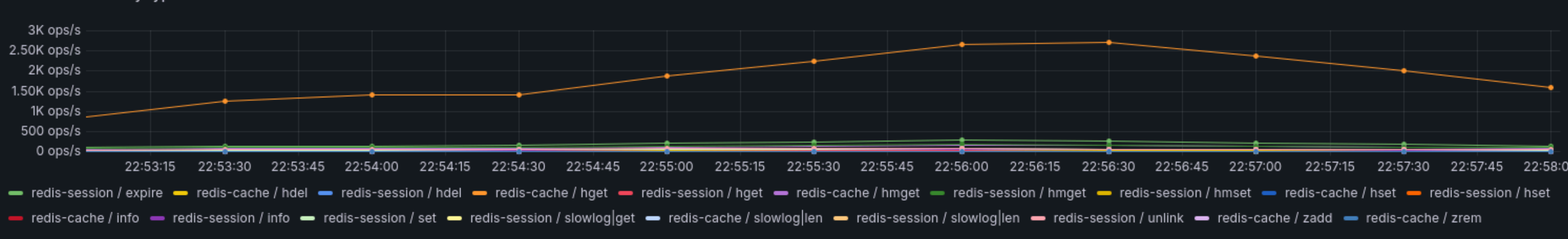
Memory used vs maxmemory by role



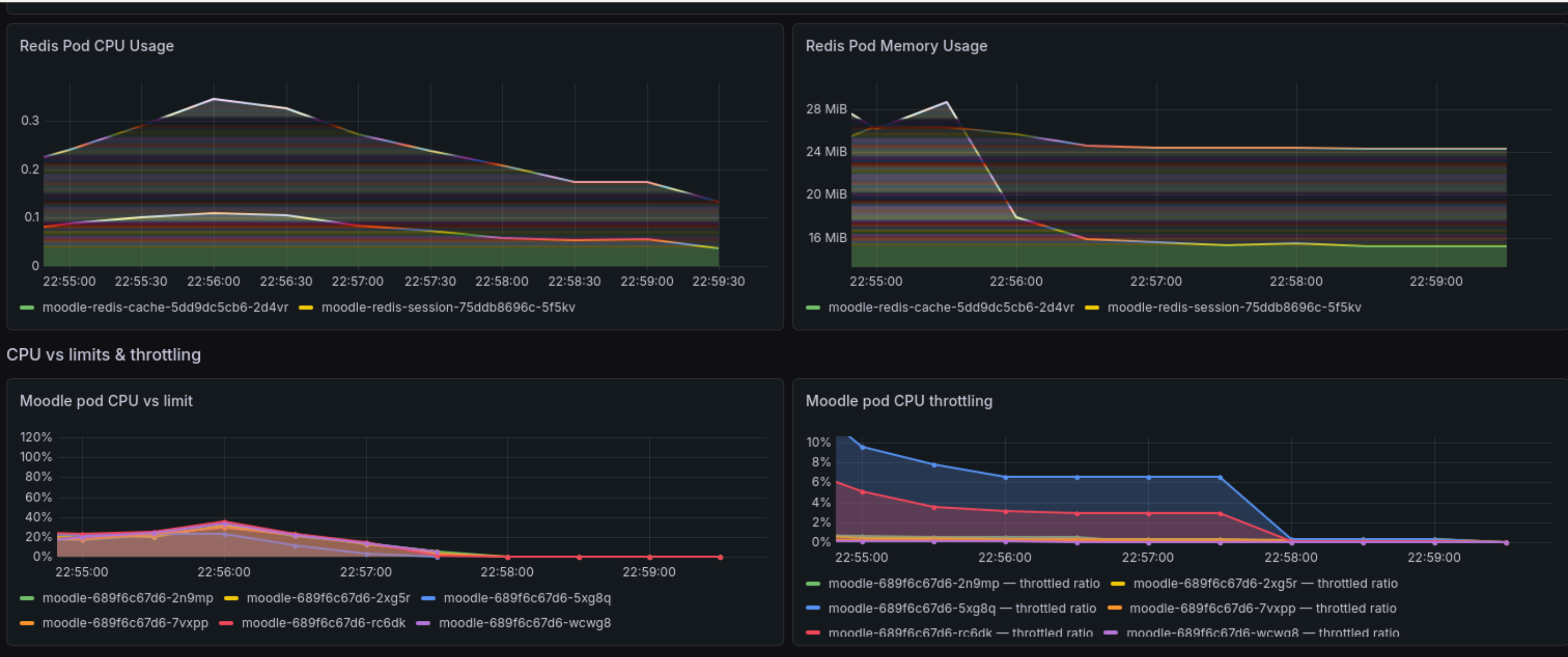
Connected / blocked clients by role



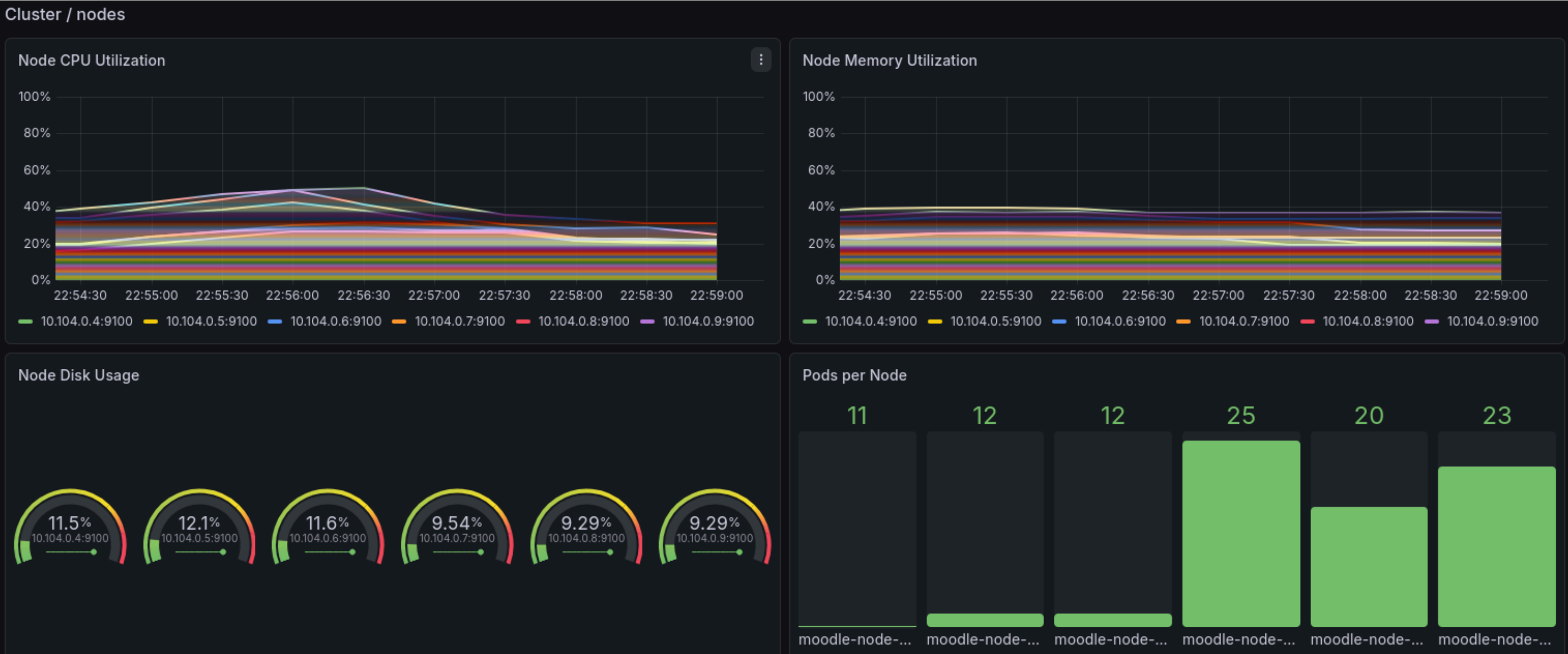
Commands rate by type and role



Dashboard Grafana



Dashboard Grafana



Hạn chế

- Hệ thống chỉ mới kiểm thử ngắn hạn, thực tế khi chạy dài hạn sẽ gặp nhiều vấn đề về hạ tầng như tràn bộ nhớ, rò rỉ dữ liệu.
(vd: tràn cache làm mất phiên đăng nhập người dùng), ...
- Quy trình tự động hóa đang dừng lại ở CI/CD app, hạ tầng. Vẫn còn nhiều vấn đề như tự động hóa quét bảo mật, kiểm soát chi phí, backup dữ liệu, ...
- Sử dụng full tính năng AI của moodle (Image, Video) cần server cực mạnh, rất khó đáp ứng được toàn bộ người dùng LMS.

Hướng phát triển

- Thực hiện các bước bảo vệ, thông báo khi có sự cố kỹ càng hơn.
- Logging để lưu trữ thông tin cho hoạt động hệ thống hằng ngày.
- Waiting Room cho Cloudflare để giảm thời gian downtime.
- Can thiệp sâu hơn về layer software (tối ưu database query, api,...) thay vì chỉ can thiệp ở layer infra như hiện tại (caching, proxy,...)

5

DEMO



**Thank You For
Your Listening**